

Technical Report: From RGB Matrices to Scaled Black & White Images

Image Processing, Matrix Representation, $YCbCr$
Formats, Static Reference Thresholds, and Adaptive
Scaling Algorithms

Prepared by:

Pranay

References & Citations:

1. ITU-R Recommendation BT.601: Digital Television Studio Encoding Standards.
2. W3C Web Content Accessibility Guidelines (WCAG) 2.1: Luminance Metrics.
3. Python Imaging Library (Pillow) Documentation: Image Modes (RGB, L, 1).

September 24, 2026

1 Raw RGB Matrices vs. Compiled JPEG Matrices

A digital image in its raw form is stored directly as a 2D matrix of pixel elements.

1.1 Raw RGB Format

In a standard *RGB* image of width W and height H , the image is stored as a 2D matrix where every element at coordinates (x, y) is a **3-element tuple**:

$$\text{Pixel}(x, y) = (R_{x,y}, G_{x,y}, B_{x,y})$$

where $R, G, B \in [0, 255]$.

An uncompressed *RGB* image memory structure looks like this:

$$\mathbf{M}_{RGB} = \begin{bmatrix} (R_{0,0}, G_{0,0}, B_{0,0}) & (R_{1,0}, G_{1,0}, B_{1,0}) & \dots & (R_{W-1,0}, G_{W-1,0}, B_{W-1,0}) \\ (R_{0,1}, G_{0,1}, B_{0,1}) & (R_{1,1}, G_{1,1}, B_{1,1}) & \dots & (R_{W-1,1}, G_{W-1,1}, B_{W-1,1}) \\ \vdots & \vdots & \ddots & \vdots \\ (R_{0,H-1}, G_{0,H-1}, B_{0,H-1}) & \dots & \dots & (R_{W-1,H-1}, G_{W-1,H-1}, B_{W-1,H-1}) \end{bmatrix}$$

1.2 Compiled JPEG Matrix Structure

When an image is compiled into a JPEG format using 4:2:0 subsampling, it is split into two primary layers:

1. **The Luminance Matrix (Y):** A full $W \times H$ matrix containing brightness data for every single pixel.
2. **The Chrominance Matrix (Cb/Cr):** A smaller $\frac{W}{2} \times \frac{H}{2}$ matrix where each element contains a **shared 2×2 color block**.

Every 2×2 block of 4 adjacent pixels shares a single Cb and Cr color pair while retaining 4 individual Y brightness values:

$$\text{Block}_{2 \times 2} = \begin{bmatrix} Y_{0,0} & Y_{1,0} \\ Y_{0,1} & Y_{1,1} \end{bmatrix} \quad \text{with a single shared color } (Cb_{\text{avg}}, Cr_{\text{avg}})$$

—

2 Calculating and Storing Y , Cb , and Cr

To separate brightness from color, raw *RGB* values are transformed using the standard ITU-R BT.601 conversion formulas:

2.1 Mathematical Formulas

$$\begin{aligned} Y &= 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B \\ Cb &= 128 - 0.168736 \cdot R - 0.331264 \cdot G + 0.5 \cdot B \\ Cr &= 128 + 0.5 \cdot R - 0.418688 \cdot G - 0.081312 \cdot B \end{aligned}$$

2.2 Storage Principles

- **Y Channel Range:** $0 \leq Y \leq 255$ (0 is pitch black, 255 is pure white).
 - **Cb and Cr Range:** $0 \leq Cb, Cr \leq 255$ (The +128 offset prevents negative values; 128 represents neutral gray/no color).
-

3 Grayscale Conversion: From Tuple Array to Scalar Array

Converting a color image to grayscale involves discarding all color channels and keeping **only the Y values**.

3.1 Data Structure Difference

- **RGB Format:** An array of 3-element tuples $\rightarrow [(R_1, G_1, B_1), (R_2, G_2, B_2), \dots]$
- **Grayscale Format:** A 2D array of **single scalar numbers** $\rightarrow [Y_1, Y_2, Y_3, \dots]$

The resulting Grayscale matrix is simply:

$$\mathbf{M}_{\text{Gray}} = \begin{bmatrix} Y_{0,0} & Y_{1,0} & \dots & Y_{W-1,0} \\ Y_{0,1} & Y_{1,1} & \dots & Y_{W-1,1} \\ \vdots & \vdots & \ddots & \vdots \\ Y_{0,H-1} & \dots & \dots & Y_{W-1,H-1} \end{bmatrix}$$

4 Thresholding Algorithms: Static Reference vs. Adaptive Scaling

Binary black-and-white conversion transforms a 2D scalar array into a binary matrix (0 or 255).

4.1 Static 128 Reference Threshold (The "Reference Breaker")

The naive approach uses a static midpoint threshold $T = 128$:

$$\text{StaticPixel}(x, y) = \begin{cases} 255 & \text{if } Y_{x,y} \geq 128 \quad (\text{White}) \\ 0 & \text{if } Y_{x,y} < 128 \quad (\text{Black}) \end{cases}$$

Limitation: If an image is underexposed ($Y < 128$ across most pixels), the entire output collapses into pure black. Conversely, overexposed images collapse into pure white.

4.2 Adaptive Scaled Black & White Algorithm

To prevent thresholding failure, the threshold is dynamically computed using the image's mean intensity (μ):

$$\mu = \frac{1}{W \cdot H} \sum_{x=0}^{W-1} \sum_{y=0}^{H-1} Y_{x,y}$$
$$\text{AdaptivePixel}(x, y) = \begin{cases} 255 & \text{if } Y_{x,y} \geq \mu \quad (\text{White}) \\ 0 & \text{if } Y_{x,y} < \mu \quad (\text{Black}) \end{cases}$$

5 Python Implementation

The code below extracts raw *RGB* arrays, performs *YCbCr* transformation, extracts *Y* scalar arrays, and generates both `output_ref.png` (static reference) and `output_adaptive_bw.png` (adaptive threshold).

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def load_jpg_as_rgb(image_path):
5     try:
6         import imageio.v3 as iio
7         rgb_matrix = iio.imread(image_path)
8     except ImportError:
9         rgb_matrix = plt.imread(image_path)
10    return rgb_matrix.astype(np.float32)
11
12 def rgb_to_ycbcr(rgb_matrix):
13    R = rgb_matrix[:, :, 0]
14    G = rgb_matrix[:, :, 1]
15    B = rgb_matrix[:, :, 2]
16
17    Y = 0.299 * R + 0.587 * G + 0.114 * B
18    Cb = 128 - 0.168736 * R - 0.331264 * G + 0.5 * B
19    Cr = 128 + 0.5 * R - 0.418688 * G - 0.081312 * B
20
21    return np.dstack((Y, Cb, Cr))
22
23 def ycbcr_to_grayscale(ycbcr_matrix):
24    grayscale_matrix = ycbcr_matrix[:, :, 0]
25    return np.clip(grayscale_matrix, 0, 255).astype(np.uint8)
26
27 def grayscale_to_static_bw(grayscale_matrix, threshold=128):
28    # Static Reference Breaker (Fixed at 128)
29    return np.where(grayscale_matrix >= threshold, 255, 0).astype(np.uint8)
30
31 def grayscale_to_adaptive_bw(grayscale_matrix):
32    # Adaptive Dynamic Thresholding
33    mu = np.mean(grayscale_matrix)
34    return np.where(grayscale_matrix >= mu, 255, 0).astype(np.uint8)
```

Listing 1: Image Processing and Thresholding Pipeline